

Trabajo Final Inteligencia Artificial 1 año 2025: Clasificador de piezas mediante visión artificial controlado por reconocimiento de voz.

García Sad Tomás Agustín

Facultad de Ingeniería de la Universidad Nacional de Cuyo

Inteligencia Artificial 1

6 de noviembre de 2025

Resumen

Se desarrolló un agente de software para resolver un problema de clasificación y decisión en un entorno de producción simulado. El objetivo es estimar la proporción de piezas (tornillos, tuercas, arandelas y clavos) dentro de una caja, basándose en una muestra de 10 unidades y controlado por comandos de voz.

Para la identificación visual de la muestra se implementó el algoritmo K-Means, utilizando descriptores de forma como atributos. Para la estimación de la caja se utilizó un Clasificador Bayesiano Multinomial, demostrando su capacidad de actualización secuencial. Finalmente, el reconocimiento de los comandos ("Contar", "Proporción", "Salir") se realizó con un algoritmo K-NN implementado sobre características MFCC.

En las pruebas de validación, el clasificador K-Means demostró una alta separabilidad de las clases, logrando que los 4 conjuntos generados se mapearan correctamente a las 4 categorías de piezas. El K-NN mostró una distinción robusta entre los comandos de voz. El estimador Bayesiano funcionó según lo esperado, actualizando secuencialmente las probabilidades y convergiendo a la hipótesis de caja más coherente con la muestra de piezas escaneada.

Se concluye que el agente integra exitosamente las técnicas de IA, logrando una solución robusta y viable para el problema planteado.

Índice

Contenido

Resumen	2
Índice	3
Introducción.....	4
Visión Artificial	4
Reconocimiento de Voz	4
Descripción del Problema a Resolver	4
Especificación del agente	4
Tipo de Agente	4
Tabla REAS.....	5
Propiedades del Entorno.....	5
Diseño del agente	6
1. Flujo de Visión (K-Means).....	6
2. Flujo de Voz (K-NN)	7
3. Módulo de Decisión (Estimación Bayesiana)	8
Código	9
Ejemplo de aplicación	10
1. Ejemplo de Procesamiento de Visión (K-Means)	10
2. Ejemplo de Procesamiento de Voz (K-NN)	11
3. Ejemplo de Decisión (Aprendizaje Bayesiano).....	12
Resultados.....	13
1. Resultados del Clasificador de Visión (K-Means)	13
2. Resultados del Clasificador de Voz (K-NN)	14
3. Resultados del Estimador Bayesiano	15
Conclusiones	15
Bibliografía	17
IA generativa.....	17

Introducción

La Inteligencia Artificial (IA) ha permitido el desarrollo de sistemas capaces de percibir su entorno y tomar decisiones. Este proyecto integra tres dominios clave de la IA para resolver un problema de clasificación en un entorno de producción: la visión artificial, el reconocimiento de voz y la inferencia probabilística.

Visión Artificial

La visión artificial es el campo de la IA que permite a las computadoras y sistemas "ver" e interpretar el mundo visual. A diferencia de una cámara que simplemente graba, un sistema de visión artificial analiza la información visual para extraer datos significativos. Esto se logra procesando imágenes para identificar objetos, patrones y formas, traduciéndolos a características numéricas. En este trabajo, la visión artificial se aplica para la clasificación de piezas industriales (tornillos, tuercas, arandelas y clavos). Se utiliza el algoritmo K-Means sobre descriptores de forma extraídos de las imágenes para agrupar e identificar automáticamente cada pieza de una muestra.

Reconocimiento de Voz

El reconocimiento de voz es la tecnología que permite a una máquina recibir y procesar la voz humana. Actúa como un puente entre el lenguaje hablado y los comandos digitales, permitiendo una interacción hombre-máquina natural y manos libres. Para lograrlo, la señal de audio se convierte en un vector de características (como los MFCCs) que capturan las propiedades únicas del sonido. En este proyecto, se utiliza el algoritmo K-NN para reconocer comandos de voz específicos ("Contar", "Proporción", "Salir"), permitiendo al usuario operar el agente sin necesidad de una interfaz física.

Descripción del Problema a Resolver

El problema central de este trabajo es desarrollar un agente de software que pueda estimar la composición de una caja de 1,000 piezas basándose únicamente en una muestra aleatoria de 10 piezas. El sistema primero debe utilizar el pipeline de visión artificial (K-Means) para identificar y contar las piezas de la muestra. Luego, utilizando esos conteos, debe aplicar un clasificador Bayesiano para inferir cuál de las cuatro configuraciones de caja predefinidas (a, b, c o d) es la más probable. Todo el sistema se controla mediante los comandos de voz ("Contar", "Proporción") reconocidos por el agente.

Especificación del agente

La especificación del agente define el tipo de agente, su entorno y cómo interactúa con él, siguiendo el esquema REAS (Rendimiento, Entorno, Actuadores, Sensores) y las propiedades del entorno.

Tipo de Agente

El agente implementado es un sistema híbrido que combina dos arquitecturas definidas en la teoría:

1. Agente Reactivo Basado en Modelos: El agente no reacciona solo a la percepción actual. Mantiene un estado interno (la cola ultimas_piezas en app.py) que le permite "seguir el rastro" de las piezas que ya ha clasificado. Este estado (el historial de percepciones) es fundamental para la estimación Bayesiana.
2. Agente Basado en Objetivos: Cuando el agente recibe el comando "Proporción", no actúa solo con reglas. Consulta su información de objetivos (las cuatro definiciones de cajas en estimador_bayesiano.py). Utiliza esta información para "planificar" una acción (el cálculo Bayesiano) que le permita alcanzar el objetivo de determinar la caja más probable.

Tabla REAS

Rendimiento	Entorno	Actuadores	Sensores
Racionalidad: El agente busca ser un agente racional, es decir, tomar la acción que maximice su medida de rendimiento. En este caso, la medida de rendimiento es la convergencia exitosa: la capacidad de que su estimación de probabilidad converja a la hipótesis (caja) más coherente con la muestra de piezas disponible(evidencia)	Entorno de producción simulado. El agente interactúa con el usuario a través de una interfaz web y muestra sus resultados de razonamiento tanto en la consola del servidor como en la interfaz previamente mencionada	Los dispositivos con los que el agente actúa en su entorno son: • Salida JSON: Envía el resultado de la clasificación y el texto de los comandos a la interfaz web. • Salida en Consola: Imprime la tabla de evolución de probabilidades. • Controlador Gráfico: Ejecuta la llamada a matplotlib para mostrar el gráfico en la pantalla del servidor.	Dispositivos con los que el agente percibe el entorno son: • Sensor de Imagen (Web): El endpoint /predict_image que recibe el archivo de la foto del usuario. • Sensor de Audio (Web): El endpoint /predict_command que recibe el archivo de audio del usuario.

Propiedades del Entorno

El entorno de este agente se clasifica de la siguiente manera:

- Parcialmente Observable: El agente no puede ver el estado completo del mundo (la caja de 1.000 piezas). Sus sensores solo le dan acceso a una muestra de 10 piezas.
- Estocástico: El entorno no es determinista. El proceso de "extraer una muestra" es aleatorio. Una misma caja (ej: Caja A) no siempre producirá la misma muestra de 10 piezas.

- Secuencial : Aunque la clasificación de una foto es episódica, el núcleo del agente (la estimación Bayesiana) es secuencial. La decisión final depende de la secuencia completa de piezas observadas; la acción (el cálculo) no es independiente de las percepciones previas.
- Estático : El entorno es estático porque la caja (el mundo) no cambia sus proporciones mientras el agente está deliberando.
- Discreto: El entorno es discreto. Las percepciones (4 tipos de piezas), las acciones (3 comandos de voz) y los estados del mundo (4 tipos de cajas) son un conjunto finito y definido.
- Agente Individual: Es un agente individual. Aunque el usuario (operario) interactúa con él, no se considera un "multiagente" ya que el usuario no está compitiendo ni cooperando como otro agente racional que maximiza su propio rendimiento.

Diseño del agente

El diseño de este agente sigue el modelo de Reconocimiento de Patrones, el cual consta de una serie de etapas para procesar la información del entorno (imágenes y audio) y transformarla en una decisión final.

El sistema se divide en tres subprocessos principales:

1. Flujo de Visión (Clasificación de Piezas)
2. Flujo de Voz (Reconocimiento de Comandos)
3. Módulo de Decisión (Estimación Bayesiana)

1. Flujo de Visión (K-Means)

Este proceso es el encargado de responder a la pregunta fundamental: "¿Qué pieza estoy viendo?". Sigue las etapas clásicas del reconocimiento de patrones, un pilar de la percepción en inteligencia artificial. El flujo inicia con la Adquisición y Acondicionamiento, donde el agente "percibe" la imagen "bruta" a través del sensor web (implementado en `app.py`). Esta imagen pasa inmediatamente a un Preprocesamiento (`procesado_img.py`), donde se aplican filtros `GaussianBlur` para limpiar el ruido, se aplana el fondo para eliminar sombras y variaciones de luz, y se normaliza el tamaño de la imagen para estandarizar la entrada.

La siguiente etapa es la Segmentación (`procesado_img.py`), cuyo objetivo es aislar la pieza del fondo, es decir, identificar los elementos individuales de interés. Utilizando la imagen preprocesada, se aplica una umbralización adaptativa (`adaptiveThreshold`) para binarizarla. Posteriormente, se emplean técnicas de detección de contornos, relleno de huecos (`floodFill`) y morfología matemática para generar una máscara binaria limpia que representa únicamente la silueta de la pieza.

Una vez segmentada la pieza, el sistema aplica la Extracción de Características (momentos.py). Es crucial destacar que el análisis no se realiza sobre la imagen, sino sobre un vector de características numéricas que describen la forma de la máscara. Las características seleccionadas por su poder discriminativo son los Momentos de Hu (calculando los primeros seis), que actúan como una "huella digital" de la forma al ser invariantes a rotación y escala; la Relación de Aspecto (ar2), fundamental para diferenciar piezas alargadas (clavos, tornillos) de piezas redondas (tuercas, arandelas).

Con este vector de características, se procede a la Clasificación (Mi_Kmeans.py). Se emplea el algoritmo de aprendizaje no supervisado K-Means, ya que su objetivo es encontrar la estructura subyacente en los datos (agruparlos). El proceso de K-Means se optimizó significativamente en su Inicialización: en lugar de seleccionar centroides aleatorios, el algoritmo utiliza las 4 clases conocidas. Calcula el "medoide" de cada clase (el punto real más cercano a la media de ese grupo) y lo utiliza como centroide inicial. Luego, el algoritmo de Agrupamiento converge, organizando los datos de entrenamiento en 4 conjuntos óptimos. Una nueva pieza se clasifica calculando su vector de características y asignándola al conjunto cuyo centroide esté más cercano.

Finalmente, la etapa de Interpretación (Mi_Kmeans.py y app.py) traduce el resultado algorítmico en una decisión. El agente mapea el conjunto resultante (ej: "Conjunto 2") a un concepto del mundo real (la clase, ej: "Tornillo"), utilizando la función assign_clusters_one_to_one. Este resultado ("Tornillo") es enviado al app.py y almacenado en el estado interno del agente (la cola ultimas_piezas), quedando disponible para el módulo de decisión Bayesiano.

2. Flujo de Voz (K-NN)

Este proceso es el encargado de responder a la pregunta: "¿Qué comando dijo el usuario?". Al igual que el pipeline de visión, sigue un flujo de Reconocimiento de Patrones. La etapa de Adquisición y Acondicionamiento comienza cuando el agente "percibe" el audio "bruto" desde el micrófono del navegador (app.py). Este audio pasa inmediatamente por un Preprocesamiento (procesar_audio.py) para estandarizar la señal de entrada: se convierte a mono, se remuestrea a 16kHz, se aplican filtros pasa-banda para eliminar ruidos fuera del espectro de la voz humana, se eliminan los silencios al inicio y al final, y se normaliza la amplitud para que el volumen de la grabación no afecte al resultado.

Una vez obtenida la señal limpia, se aplica la Segmentación (cualidades_audio.py), donde el audio se divide en 10 segmentos temporales iguales. Esto permite analizar cómo evolucionan las características de la palabra a lo largo del tiempo. A continuación, la etapa de Extracción de Características (cualidades_audio.py) calcula un vector de 140 características numéricas que describe la palabra completa. Las características principales seleccionadas fueron los Coeficientes Cepstrales en Frecuencia Mel (MFCCs), que son el estándar en reconocimiento de voz por su capacidad de imitar la percepción no lineal del oído humano; la Tasa de Cruce por Cero (ZCR), útil para

detectar sonidos sibilantes como la 'S' en "Salir"; y la Energía (RMS), que mide la intensidad de cada segmento.

Finalmente, se realiza la Clasificación (`predecir_audio.py`). Para esta tarea se seleccionó el algoritmo de aprendizaje supervisado K-NearNeighbors (K-NN). El "entrenamiento" de K-NN es directo: consiste simplemente en almacenar los vectores de 140 características de todos los audios de ejemplo (con sus etiquetas "Contar", "Proporción" o "Salir"). En la fase de clasificación, el algoritmo mide la distancia entre el nuevo vector de características y todos los vectores almacenados. Mediante una Votación, se seleccionan los 7 vecinos más cercanos, y el comando se decide por la etiqueta más frecuente entre ellos. La etapa de Interpretación ocurre en `app.py`, que recibe la clase resultante (ej: "Proporción") y ejecuta la acción correspondiente en el agente.

3. Módulo de Decisión (Estimación Bayesiana)

Esta es la etapa de razonamiento principal del agente, donde se conectan los dos flujos de percepción (visión y voz). El algoritmo seleccionado es el Aprendizaje Bayesiano. Este enfoque permite al agente actualizar su "grado de creencia" inicial sobre una hipótesis a medida que recibe nueva evidencia. En este proyecto, las "hipótesis" (H) son las 4 posibles configuraciones de cajas (A, B, C, D), y la "evidencia" (D) es cada pieza individual clasificada por el flujo de visión.

El diseño de este módulo, implementado en `estimador_bayesiano.py`, sigue de cerca el modelo teórico de actualización de probabilidades. El agente inicia con un conocimiento previo (A Priori), que son las 4 definiciones de cajas y una probabilidad a priori uniforme ($P(H) = 0.25$ para cada una), reflejando la falta de evidencia inicial. El agente implementa una actualización continua: por cada pieza observada y almacenada en la memoria (`ultimas_piezas`), recalcula la probabilidad a posteriori ($P(H/D)$) de las 4 cajas usando el Teorema de Bayes.

Para asegurar la estabilidad numérica, el sistema evita la multiplicación directa de probabilidades, que llevaría a un underflow (números demasiado pequeños para ser representados). En su lugar, todos los cálculos se realizan en el espacio logarítmico. La multiplicación de probabilidades se convierte así en una suma de log-probabilidades, un método mucho más robusto. Este enfoque también permite un manejo elegante de la evidencia imposible; si el agente observa una "Arandela", la verosimilitud de la "Caja D" (que tiene 0 arandelas) se calcula como $\log(0)$, resultando en $-\infty$. Esto elimina automáticamente esa hipótesis de las posibilidades.

Finalmente, la etapa de Interpretación o decisión ocurre al final del proceso. La "decisión" final del agente (la estimación que reporta al usuario) es la hipótesis (la caja) que posee la probabilidad más alta.

Código

El diseño del agente descrito previamente se implementa a través de un conjunto de módulos de software en Python. Esta sección menciona cuales son los fragmentos de código más relevantes que implementan la lógica central del sistema, demostrando cómo se aplican los algoritmos de K-Means, K-NN y Aprendizaje Bayesiano.

El código fuente completo y detallado de todos los módulos desarrollados se encuentra disponible para su consulta en el anexo que se encontrará en forma de carpeta .zip. Dicho código también se puede encontrar en el repositorio <https://github.com/tomasagustingarciasad-cmyk/Final-IA>. (La ventaja del repositorio radica en que algunos códigos tienen llamadas a archivos mediante las direcciones, pero su desventaja es que en él se encuentra la base de datos, por lo que es pesado, llegando a pesar 1.2GB al momento de escribir esta sección del informe)

El núcleo del agente reside en app.py, un servidor web implementado con Flask que actúa como el "cerebro" orquestador. Este módulo es responsable de cargar los modelos de K-Means y K-NN en memoria, mantener el estado interno del agente (la cola ultimas_piezas) y gestionar los puntos de entrada (endpoints) para la percepción visual y auditiva.

El componente de razonamiento probabilístico se implementa en estimador_bayesiano.py. Este módulo es el corazón de la lógica de decisión del agente. La función calcular_evolucion_probabilidades implementa la actualización Bayesiana secuencial. Esta función itera sobre cada pieza observada y actualiza el "grado de creencia" de las cuatro hipótesis (cajas).

Para la clasificación de imágenes, el algoritmo K-Means se implementó desde cero en Mi_Kmeans.py. Un aspecto fundamental de su diseño es la inicialización de los centroides. En lugar de una selección aleatoria (que es sensible a los valores iniciales), se implementó un método basado en prototipos de clase. La función build_class_prototypes calcula el "medoide" de cada una de las 4 clases conocidas; es decir, selecciona el punto real de los datos de entrenamiento que es más cercano a la media de esa clase. Esto asegura una inicialización de centroides consistente y representativa, mejorando la calidad del agrupamiento final.

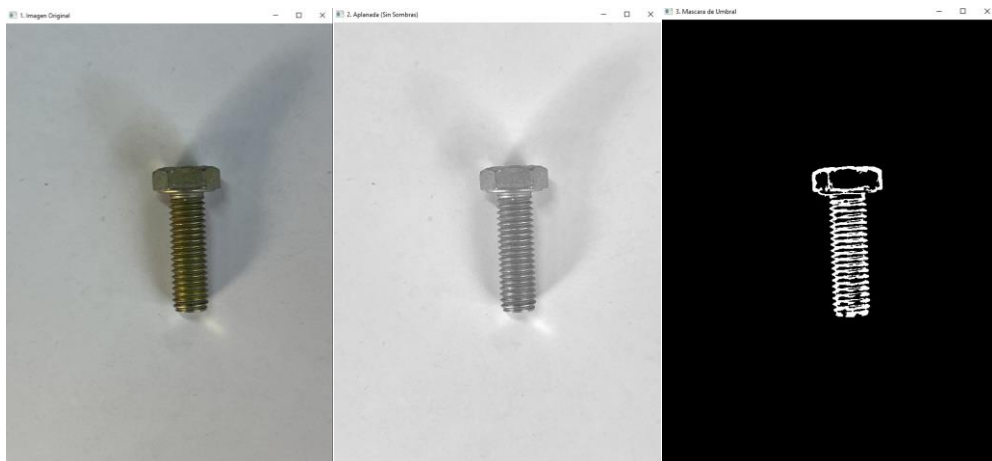
Finalmente, el reconocimiento de voz se implementa en predecir_audio.py mediante el algoritmo K-NN, un método de aprendizaje supervisado. La función clave predecir_knn implementa el clasificador. Recibe el vector de 140 características del audio nuevo y calcula la distancia contra todos los vectores de la base de entrenamiento. La clasificación final se determina por una "votación" de los 'K' vecinos más cercanos, asignando la clase más frecuente (ej: "Contar", "Proporción" o "Salir").

Ejemplo de aplicación

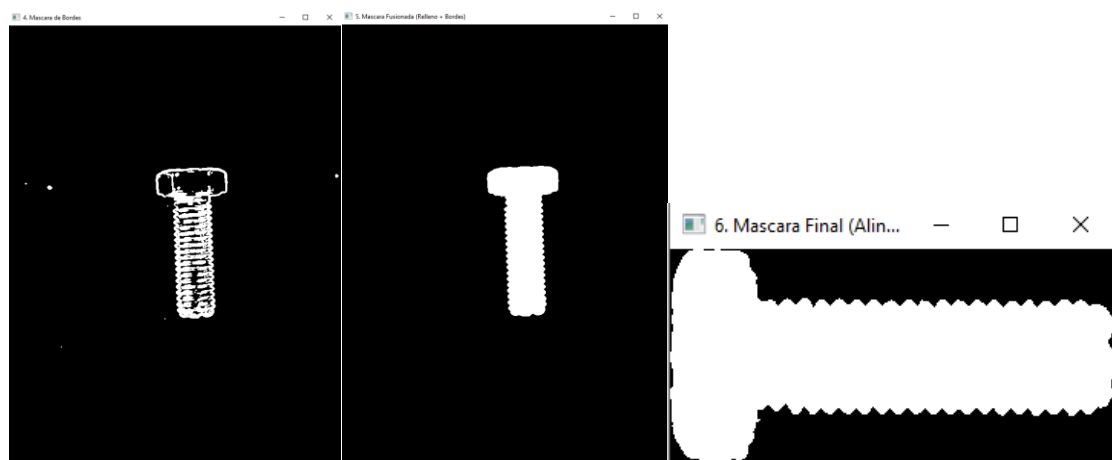
Para demostrar el funcionamiento de los algoritmos descritos en el diseño, esta sección presenta un ejemplo concreto del procesamiento de la información en cada uno de los pipelines principales del agente.

1. Ejemplo de Procesamiento de Visión (K-Means)

El primer ejemplo sigue a una imagen de un tornillo, a través de todo el pipeline de visión. El proceso inicia con la Adquisición y Segmentación, donde la imagen original es procesada por el script `procesado_img.py`. Este módulo limpia, binariza y alinea la imagen, generando como salida una máscara binaria que aísla perfectamente la silueta de la pieza.



Imágenes 1, 2 y 3: Visualización de los procesos para la obtención de imagen: Imagen original, Imagen en escala de grises, binarización corte por umbral.



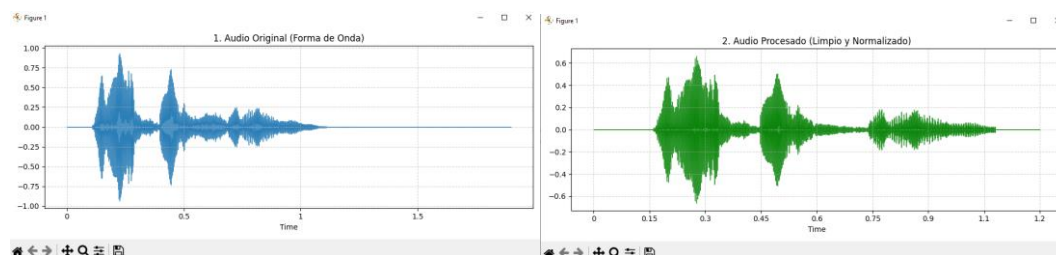
Imágenes 4, 5 y 6: Visualización de los procesos para la obtención de imagen: Binarización con bordes engrosados, binarización rellena, binarización con recorte y alineado.

A continuación, esta máscara pasa a la etapa de Extracción de Características (momentos.py). El script calcula el vector de características numéricas que describe su forma y guarda todos los parámetros extraídos, tales como file, clase, hu1, hu2, hu3, hu4, hu5, hu6, **circularidad**, redondez, aspect_ratio, ar2, n_lados, grad_mean, edge_density. Para este tornillo, el vector de las características es Tornillo018.png, Tornillo, 0.303033, 1.622040, 3.541322, 3.074238, 6.415762, 3.901736, **0.203127**, 0.145010, 0.259690, 0.477663, 4, 5.311604, 0.000000. Nótese el valor bajo de ar2, que indica una forma alargada.

Finalmente, este vector llega a la etapa de Clasificación. El script predecir_imagenes.py carga el modelo K-Means (Mi_Kmeans.py). El vector es estandarizado y se calcula su distancia a los centroides de los 4 conjuntos aprendidos. El vector del tornillo es asignado al conjunto más cercano (ej: "Conjunto 3"), que la función assign_clusters_one_to_one interpreta y devuelve como la clase "Tornillo".

2. Ejemplo de Procesamiento de Voz (K-NN)

El segundo ejemplo sigue un comando de voz, como "Proporción". El audio original es procesado por procesar_audio.py, donde se aplican filtros, se recortan los silencios y se normaliza la amplitud, tal como se describió en el diseño.



Imágenes 7 y 8: Visualización de los procesos para la obtención de audio: Audio Prop_Est.mp4 en formato original y audio con procesamiento completo.

Este audio limpio pasa al script cualidades_audio.py, que realiza la Extracción de Características dividiendo la señal en 10 segmentos y calculando las 140 características (MFCCs, ZCR, RMS). El vector resultante es enviado al clasificador predecir_audio.py. Este módulo implementa K-NN: calcula la distancia del nuevo vector contra todos los vectores de la base de entrenamiento y realiza la votación. El resultado de la votación es la clase "Proporcion".

Salida por consola:

[ETAPA 3: Extracción de Características]

Vector de características extraído.

-> Dimensiones del vector: (140,)

-> Primeras 5 características (de 140): [1.1225000e-01 5.6232769e-02 -5.0315210e+02 -2.7146246e+02 1.8787421e+02]

[ETAPA 4: Clasificación K-NN]

¡Clasificación completa!

-> Comando detectado: Proporción

-> Distancia promedio a vecinos: 11.2853

--- FIN DEL PROCESO ---

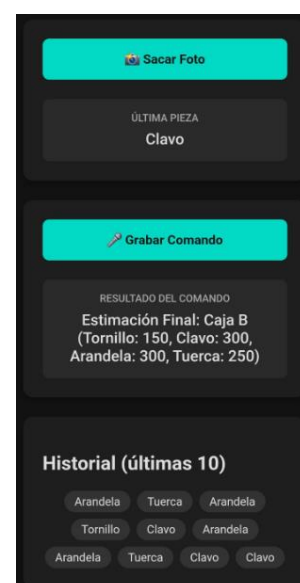
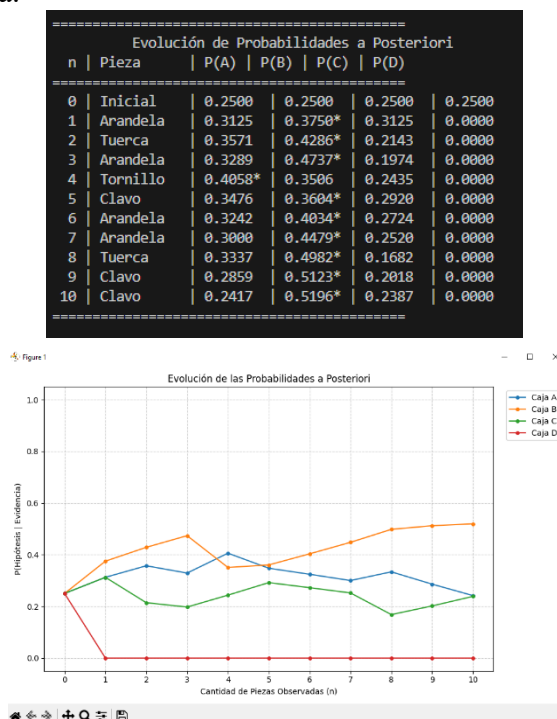
3. Ejemplo de Decisión (Aprendizaje Bayesiano)

El último ejemplo demuestra el módulo de razonamiento. Este se activa cuando el app.py recibe el comando "Proporción" del paso anterior. El agente consulta su estado interno (la memoria de 10 piezas) que fue llenado por el pipeline de visión.

Supongamos que el estado interno es: ['Arandela', 'Tuerca', 'Arandela', 'Tornillo', 'Clavo', 'Arandela', 'Arandela', 'Tuerca', 'Clavo', 'Clavo']. Esto genera un conteo de muestra de: {Tornillo: 1, Clavo: 3, Arandela: 4, Tuerca: 2}.

El módulo estimador_bayesiano.py toma este historial y realiza la actualización secuencial de probabilidades. En la consola del servidor se imprime la tabla de evolución, mostrando cómo las probabilidades (grados de creencia) convergen pieza a pieza.

Simultáneamente, se genera el gráfico de convergencia en una ventana local. Este gráfico demuestra visualmente cómo el agente "aprende" con cada evidencia. En este ejemplo, las probabilidades de "Caja B" y "Caja A" "competirían", mientras que la "Caja D" sería descartada en el momento en que se observa la primera Arandela. La estimación final del agente es la hipótesis (Caja B) con la probabilidad más alta al final de la secuencia.



Imágenes 9, 10 y 11: Salida en la consola del servidor tras el comando 'Proporción'. La tabla muestra la actualización Bayesiana e interfaz de la web con la petición.

Resultados

En esta sección se presentan las especificaciones técnicas de los datos utilizados para el entrenamiento y la evaluación de los tres algoritmos principales del agente: K-Means (visión), K-NN (voz) y el estimador Bayesiano (decisión).

1. Resultados del Clasificador de Visión (K-Means)

Datos Utilizados:

La base de datos para la percepción visual se construyó siguiendo los requisitos de la consigna, tomando más de 6 fotografías de cada una de las 4 clases de piezas (Arandela, Tuerca, Tornillo, Clavo). Cada imagen fue procesada por el flujo de procesado_img.py para generar una máscara binaria. Sobre estas máscaras, el script momentos.py extrajo un vector de características de forma. Para el entrenamiento del modelo K-Means, se utilizaron las 3 características con mayor poder de separación: los dos primeros Momentos de Hu (transformados logarítmicamente, hu1_log, hu2_log) y la Relación de Aspecto (ar2).

Estadísticas de Clasificación:

Dado que K-Means es un algoritmo de aprendizaje no supervisado, el "resultado" no es una precisión tradicional, sino una medida de la pureza de los conjuntos encontrados. La estadística principal es la matriz de confusión generada por el script Mi_Kmeans.py, que compara los conjuntos encontrados por el algoritmo (Columnas) con las clases reales de los datos de entrenamiento (Filas).

Salida por consola:

Crosstab cluster×clase:

clase_norm Arandela Clavo Tornillo Tuerca

cluster

0	24	0	0	0
1	0	0	0	39
2	0	0	49	0
3	0	29	0	0

Comentario de Resultados:

La matriz de confusión demuestra una excelente separabilidad de las clases. En un escenario ideal, la tabla sería una matriz diagonal perfecta en caso de encontrarse ordenada. Como se observa en los resultados, el modelo diferencia perfectamente

Arandelas y Tuercas, asignándolas a conjuntos únicos. Se observa la misma situación entre Clavos y Tornillos, lo que nos da idea de una buena separación entre todos los elementos. No obstante, la alta pureza de los conjuntos valida el método de extracción de características y la inicialización por "medoide" como una estrategia robusta para la clasificación de estas piezas.

2. Resultados del Clasificador de Voz (K-NN)

Datos Utilizados:

La base de datos para el reconocimiento de voz se construyó a partir de un gran número de personas diferentes pronunciando los 3 comandos ("Contar", "Proporción", "Salir"). Cada grabación fue procesada por `procesar_audio.py` y `cualidades_audio.py` para generar un vector de 140 características (MFCCs, ZCR, RMS). El conjunto final de datos consiste en una matriz `X_audio.npy` (los vectores de características) y un vector `y_audio.npy` (las etiquetas de clase).

Datos de Prueba y Estadísticas:

Para evaluar el rendimiento del clasificador K-NN con los datos disponibles, se utilizó una validación cruzada. Este método consiste en entrenar el modelo con un 80% de los datos, y usar ese 20% restante como prueba. Esta técnica proporciona una estimación robusta del rendimiento del modelo en datos no vistos. Los resultados se resumen en la siguiente matriz de confusión.

Salida por consola:

Matriz de confusión:

	pred_Contar	pred_Proporcion	pred_Salir
real_Contar	5	0	0
real_Proporcion	0	5	0
real_Salir	0	0	5

Comentario de Resultados:

Los resultados de la validación cruzada muestran un rendimiento perfecto del clasificador K-NN, alcanzando una precisión general del 100%. La matriz de confusión es una diagonal ideal, lo que indica que no hubo un solo error de clasificación en todo el conjunto de datos de prueba. Este resultado sobresaliente valida de forma contundente el pipeline de procesamiento de audio: demuestra que el vector de 140 características, y en particular los MFCCs, es altamente discriminativo y permite una separación total de las clases de comandos "Contar", "Proporción" y "Salir". La elección de K-NN, combinada con esta extracción de características, se confirma como una estrategia óptima y extremadamente robusta para la tarea de control por voz del agente.

3. Resultados del Estimador Bayesiano

Datos Utilizados y de Prueba:

A diferencia de los otros algoritmos, el estimador Bayesiano no "aprende" de un conjunto de datos de entrenamiento. Su "conocimiento" se basa en el modelo a priori de las 4 cajas (A, B, C, D) definido en `estimador_bayesiano.py`. Por lo tanto, su evaluación no es estadística, sino cualitativa y observacional, diseñada para verificar si el agente razona correctamente según la teoría de probabilidad Bayesiana.

Comentario de Resultados:

La prueba del estimador se presentó en la sección anterior, "Ejemplo de aplicación". Los resultados demuestran que el agente se comporta de manera racional y correcta:

1. Convergencia: El agente actualiza su "grado de creencia" secuencialmente con cada pieza, y las probabilidades convergen hacia la hipótesis (caja) más coherente con la evidencia.
2. Descarte de Hipótesis: El agente maneja correctamente la evidencia imposible. Como se mostró en el ejemplo, al observar una "Arandela", la probabilidad de la "Caja D" (que tiene 0 Arandelas) colapsa correctamente a cero (-infinito en log-probabilidad).

El comportamiento observado en la tabla y el gráfico de convergencia valida que el módulo de decisión Bayesiano cumple exitosamente con su objetivo de diseño.

Conclusiones

Este trabajo ha permitido el diseño e implementación exitosa de un agente inteligente capaz de resolver el problema de clasificación de piezas propuesto. Se logró la integración completa de los tres pilares de la IA requeridos: la percepción (visión y voz), el aprendizaje (supervisado y no supervisado) y el razonamiento probabilístico. El sistema final es capaz de percibir su entorno mediante K-Means (para visión) y K-NN (para voz), y tomar una decisión racional utilizando un Clasificador Bayesiano.

La eficiencia de los módulos de percepción, demostrada en la sección de "Resultados", fue notable. El clasificador K-NN alcanzó una precisión del 100% en la validación, confirmando que las características de audio seleccionadas (principalmente MFCCs) son robustas y altamente separables. Del mismo modo, el agrupamiento K-Means demostró una alta pureza en los conjuntos, validando que las características de forma (Momentos de Hu y Relación de Aspecto) son suficientes para esta tarea. El módulo Bayesiano funcionó de manera lógicamente sólida, convergiendo a la hipótesis más probable y descartando correctamente las imposibles.

Entre las ventajas de este diseño, destaca la modularidad del código, que separa la percepción (`procesado_img.py`, `procesar_audio.py`), la extracción de características (`momentos.py`, `cualidades_audio.py`) y el razonamiento (`estimador_bayesiano.py`),

facilitando su mantenimiento y explicación. Otra ventaja clave fue la inicialización del K-Means, que al usar "medoides" (prototipos reales de cada clase) en lugar de un inicio aleatorio, aseguró una convergencia estable y precisa.

Sin embargo, el diseño también presenta desventajas. El flujo de visión es sensible a la calidad de la segmentación; un fallo en la máscara (ej. por mala iluminación) invalidaría la extracción de características y la clasificación. El clasificador K-NN, si bien es preciso, no es escalable; su tiempo de respuesta aumentaría linealmente si la base de datos de voz creciera a miles de muestras. Finalmente, la estimación Bayesiana se basa en una muestra de solo 10 piezas, lo que introduce una alta varianza; una muestra diferente tomada de la misma caja podría, por puro azar, llevar a una estimación incorrecta.

Bibliografía

Alcántara Rosas, A. (2020, 1 de agosto). *K-Means: Programando el algoritmo desde cero en Python*. Medium. <https://medium.com/@alcantararosas/k-means-programando-el-algoritmo-desde-cero-en-python-f633b3a1c125>

Aldo. (2020, 10 de enero). *Remove shadow/edges from image* [Publicación en foro en línea]. OpenCV. <https://forum.opencv.org/t/remove-shadow-edges-from-image/1368>

Flimzy. (2013, 13 de noviembre). *What is the problem with shadowing names defined in outer scopes?* [Publicación en foro en línea]. Stack Overflow. <https://stackoverflow.com/questions/20125172/what-is-the-problem-with-shadowing-names-defined-in-outer-scopes>

Imagina Formación. (s.f.). *Tutorial OpenCV en Python*. Imagina Formación. <https://imaginaformacion.com/tutoriales/opencv-en-python>

Ishmatuka, C. (2020, 29 de junio). *K-Nearest Neighbors (KNN) implementation using Python*. Medium. <https://cendikiaishmatuka.medium.com/k-nearest-neighbors-knn-implementation-using-python-a1ea9d89f582>

q2bstudio. (s.f.). *Tutorial de Python: sombras y contornos en texto sobre imágenes*. q2bstudio. <https://www.q2bstudio.com/nuestro-blog/27500/tutorial-de-python-sombras-y-contornos-en-texto-sobre-imagenes>

Yogesh. (2017, 20 de junio). *How to remove shadow from scanned images using OpenCV?* [Publicación en foro en línea]. Stack Overflow. <https://stackoverflow.com/questions/44752240/how-to-remove-shadow-from-scanned-images-using-opencv>

IA generativa

Prompts utilizados:

- “Quiero desarrollar un sistema de inteligencia artificial capaz de reconocer piezas mecánicas (tornillos, tuercas, arandelas y clavos) a partir de imágenes y comandos de voz. Necesito que me ayudes a diseñar una arquitectura modular en función del código que tengo actualmente.”
- “A partir de la máscara binaria obtenida, necesito calcular los momentos de Hu y métricas geométricas como circularidad, redondez, aspect ratio y número de lados. Mostrame cómo implementarlo en un script independiente llamado momentos.py.”
- "Tengo que hacer este trabajo final para IA. Tengo bastante hecho, me gustaría que me des una mano para terminar de pulirlo y que me ayudes con el informe"
- "Bien, me gustó bastante. Ahora quiero que antes de seguir analices todas las clases de teoría.. para que tengas en cuenta esa información."

- "Bien, ahí esta funcionando y el código quedo un poco mas limpio y facil de explicar en la presentación"